Let's Build GPT (Karpathy): Synthesis

Nov 5, 2026 · @Giáo sư Thịnh

The big picture

The video builds a small GPT, a model that predicts the next character, in about 200 lines of PyTorch, and trains it on Shakespeare. Andrej Karpathy's point: ChatGPT is the same idea at enormous scale, and the core code is short. This doc is my synthesis from what I know of the video, not a transcript.

The core idea: a language model reads some text and outputs a probability for every possible next token. Generating text means sampling one token from those probabilities, appending it to the input, and repeating. Training means adjusting the weights so the real next token gets a higher probability.

The path the video takes:

1. Load text and turn characters into numbers.
2. Cut the text into training chunks and build a trivial bigram model as a baseline.
3. Learn the key trick behind attention: letting each position average information from earlier positions.
4. Build self-attention (queries, keys, values), then multi-head attention.
5. Add the rest of the transformer block: feed-forward layers, residual connections, layer norm, dropout.
6. Scale up, train, and generate text that looks like Shakespeare.
7. Connect the result to ChatGPT: what it matches and what is still missing.

Who it is for: someone who knows Python and a little PyTorch. The video assumes you have seen the earlier makemore videos, which cover embeddings, loss, and training loops.

Data and tokenization

The training text is **Tiny Shakespeare**, about 1 MB (roughly 1.1 million characters) of concatenated Shakespeare plays. It is small enough to train in minutes and rich enough to show visible progress.

Tokenization means converting text into integers. The video uses the simplest scheme: one integer per unique character.

1. Collect the sorted set of unique characters in the text. There are 65 (letters, punctuation, space, newline). That count is the `vocab_size`.
2. Build two lookup tables: `stoi` (character to integer) and `itos` (integer to character).
3. `encode(s)` turns a string into a list of integers, and `decode(l)` turns it back.
4. Store the whole dataset as one long tensor of integers.

Trade-off: character-level tokens give a tiny vocabulary but very long sequences. Real models use subword tokenizers such as BPE (GPT-2 uses about 50,000 tokens), which give shorter sequences. Karpathy mentions this and covers tokenizers in a separate video.

Batches and the bigram baseline

Train/validation split: the first 90% of the text is for training, the last 10% for validation. Validation loss shows whether the model generalizes or is just memorizing.

Training chunks: the model never sees the whole text at once. It trains on random chunks of length `block_size` (the context length, 8 in the first demos). One chunk of 9 characters gives 8 training examples at once: given the first 1 character, predict the 2nd; given the first 2, predict the 3rd; and so on up to the context length. This also teaches the model to work with any context from 1 to `block_size`.

Batches: to use the GPU well, the code stacks `batch_size` independent chunks (4 in the demo, 64 in the final model) into one tensor of shape (B, T), where B is batch size and T is time (positions). Targets are the same chunks shifted by one character.

Bigram model: the simplest possible language model, used as a baseline.

1. An `nn.Embedding(vocab_size, vocab_size)` table: each input token directly looks up a row of scores for the next token.
2. Those scores are *logits*. `F.cross_entropy(logits, targets)` measures the loss. A random model starts near ln(65), about 4.17.
3. `generate()` takes the logits for the last position, applies softmax to get probabilities, samples one token with `torch.multinomial`, appends it, and repeats.
4. Training uses the AdamW optimizer (learning rate around 1e-3 for this small model) in the standard loop: get a batch, compute loss, zero gradients, backpropagate, step.

Result: loss drops to about 2.5 and the output is still gibberish. The bigram model only sees the previous character, so it cannot use any wider context. That is the motivation for attention.

The math trick behind self-attention

Goal: let each position gather information from the positions before it, but never from the future (the future is what we are trying to predict, so peeking would be cheating).

Simplest version: average the past. For position t, take the mean of the vectors at positions 0 through t. A loop does it, but slowly. The video then shows the same result in three faster ways, each closer to real attention:

1. **Matrix multiply:** build a lower-triangular matrix `tril = torch.tril(torch.ones(T, T))`, divide each row by its sum so the rows add to 1, and compute `wei @ x`. Each output row is the average of the rows of `x` up to that position.
2. **Softmax version:** start with a matrix of zeros, set the upper triangle (the future) to negative infinity with `masked_fill`, then apply softmax across each row. The negative-infinity entries become probability 0, and the rest share probability equally. Same result as before.
3. **The key step:** the zeros in step 2 are just *affinities* between positions. Instead of equal affinities, make them **data-dependent**, so a position can pay more attention to the positions that matter most to it. That is self-attention.

Why it matters: this mask-plus-softmax pattern is the heart of a GPT. The mask keeps information flowing only from past to present, and softmax turns raw scores into weights that add to 1.

Self-attention: queries, keys, values

Every token at every position produces three vectors through three separate linear layers (no bias):

- **Query:** what am I looking for?
- **Key:** what do I contain?
- **Value:** if you find me interesting, here is what I will give you.

The computation, for one head:

1. Compute affinities: `wei = q @ k.transpose(-2, -1)`. Each position's query is compared (dot product) with every position's key. A high score means "these two are relevant to each other".
2. **Scale** by `head_size ** -0.5`. Without it, large scores make softmax too sharp (nearly one-hot), which hurts training early on, especially at initialization.

3. **Mask** the future: `wei.masked_fill(tril == 0, float('-inf'))`. This is what makes it a *decoder* block.
4. Apply **softmax** across each row to get weights that sum to 1.
5. Aggregate: `out = wei @ v`. Each position gets a weighted sum of the values of the positions it attends to.

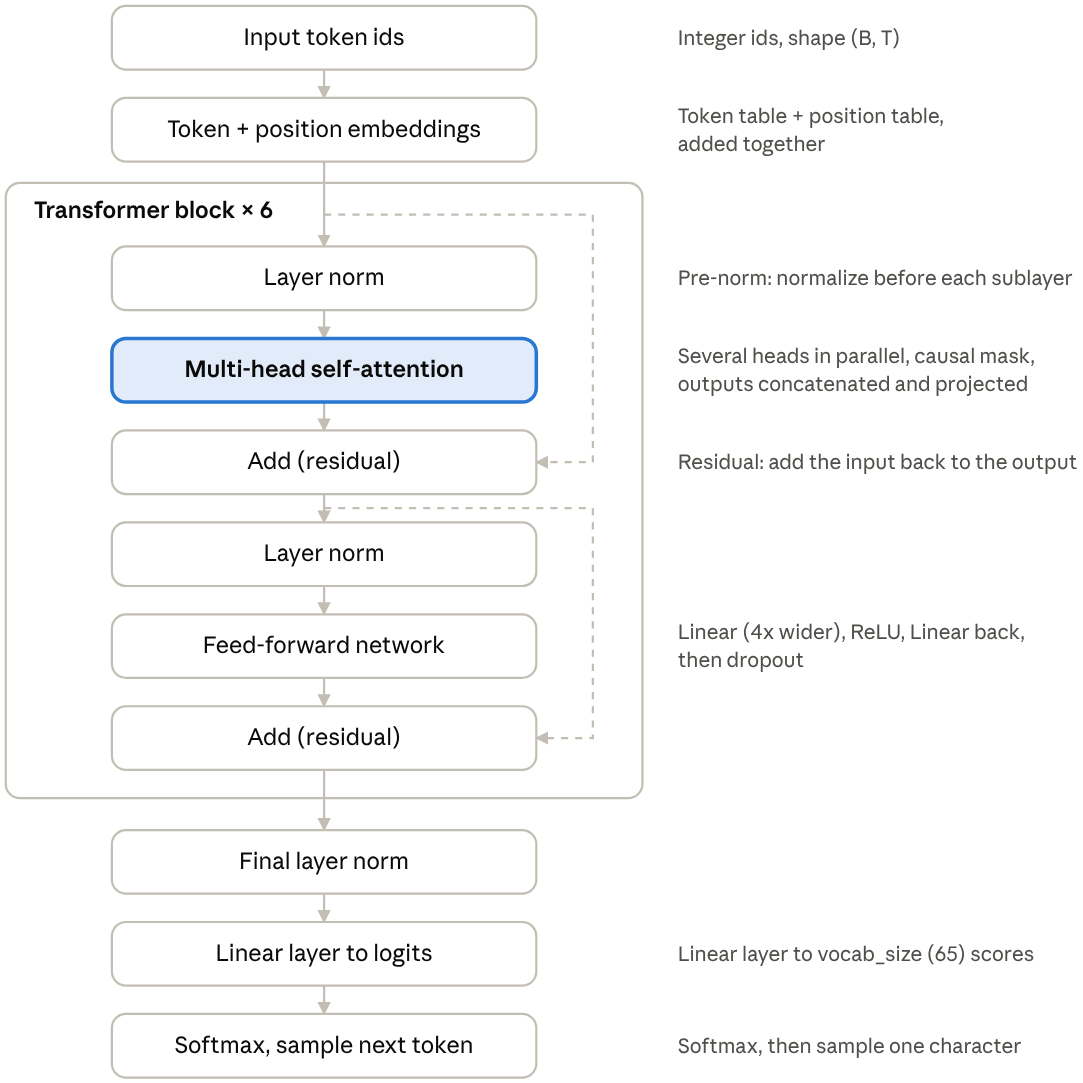
Useful facts from the video:

- Attention is a communication mechanism: nodes in a graph passing information along edges. Here the graph is "each position can see all earlier positions".
- Attention has no built-in notion of position. That is why the model adds a **position embedding** (a learned table indexed by position) to the token embedding.
- It is called *self*-attention because queries, keys, and values all come from the same input. In *cross*-attention, queries come from one place and keys and values from another (used in encoder-decoder models).
- Examples in a batch never interact. Each of the B sequences is processed independently.

Multi-head attention: run several smaller heads in parallel (for example 6 heads of size 64 instead of one of size 384), then concatenate their outputs and pass them through a linear projection. Each head can learn to look for a different kind of relationship, like having several independent channels of communication.

The transformer block

One GPT block: attention, then feed-forward, each wrapped in a residual path



GPT architecture · 1 repeated block, 2 residual paths

Read it top to bottom: tokens become vectors, pass through six identical blocks, then become next-character probabilities. Each sublayer is wrapped in a residual connection, which lets the original signal skip around it.

- **Feed-forward network:** attention lets tokens communicate; the feed-forward layer lets each token *think* about what it gathered. It is a linear layer that widens the vector 4 times, a ReLU, and a linear layer back down.

- **Residual connections:** each sublayer's output is added to its input (`x = x + sublayer(x)`). This gives gradients a direct path back through a deep network, which makes deep models trainable.
- **Layer norm:** normalizes each token's vector to zero mean and unit variance across its features. The video uses the *pre-norm* arrangement, with the norm applied before attention and before the feed-forward layer, which is a small change from the original paper.
- **Dropout:** randomly switches off some activations during training (20% in the final model) to reduce overfitting. It is applied after attention weights, after each sublayer's projection, and in the feed-forward layer.
- **Final pieces:** one more layer norm, then a linear layer that maps each position's vector to `vocab_size` logits.

Training the full model

The final script scales the earlier pieces up. The hyperparameters below are the values I recall from the video's final script (treat them as approximate).

Setting	Value	Meaning
`batch_size`	64	Independent sequences per step
`block_size`	256	Context length in characters
`n_embd`	384	Width of each token's vector
`n_head`	6	Attention heads per block (each of size 64)
`n_layer`	6	Number of stacked blocks
`dropout`	0.2	Fraction of activations randomly zeroed in training
`learning_rate`	3e-4	Lower than the bigram demo because the network is bigger
`max_iters`	5,000	Training steps

The model has roughly 10 million parameters. Karpathy trains it on a GPU in about 15 minutes.

Results: validation loss falls from about 4.2 (random guessing) to about 1.5. The generated text is now clearly Shakespeare-shaped: speaker names in capitals, line breaks, verse rhythm, and mostly real English words. It is still nonsense in meaning, because the model is tiny and saw only 1 MB of text.

Engineering details worth noting:

- `estimate_loss()` averages the loss over many batches in `eval` mode with gradients off (`torch.no_grad()`), so the printed number is stable rather than noisy.
- The training loop runs on a GPU when available (`device = 'cuda'`), with the model and every batch moved to that device.
- Samples are generated by feeding a single newline token as the starting context and calling `generate()` with a few hundred new tokens.
- During generation the context is cropped to the last `block_size` tokens, because the position embedding table has only that many rows.

From this model to ChatGPT

Decoder-only. The model built in the video has only the decoder half of the original transformer: a causal mask, no encoder, and no cross-attention. It simply continues text, which is exactly what GPT does. The original paper's encoder-decoder design suits tasks like translation, where a second input (the source sentence) conditions the output.

ChatGPT's recipe has two stages:

1. **Pretraining:** train a very large decoder-only transformer on a large share of the internet to predict the next token. This is what the video reproduces in miniature. The result is a document completer, not an assistant.
2. **Fine-tuning:** turn it into an assistant. Karpathy sketches three steps: write ideal assistant responses and train on them (supervised fine-tuning), have people rank multiple answers and train a reward model from the rankings, then optimize the model against that reward model with reinforcement learning (PPO).

What is different at scale:

	Video model	GPT-3 (as cited in the video)
Parameters	about 10 million	175 billion
Training data	about 1 million characters	about 300 billion tokens

	Video model	GPT-3 (as cited in the video)
Tokenizer	65 characters	Subword tokens (about 50,000)
Context length	256	2,048
Architecture	The same decoder-only transformer	The same, with more layers and wider layers

What is missing from the video: the fine-tuning stages, subword tokenization, and the engineering needed to train across thousands of GPUs. The architecture itself is nearly identical.

Key takeaways and a rebuild checklist

Takeaways

1. A GPT is a next-token predictor. Everything else is machinery to make that prediction use context well.
2. Self-attention is a weighted average over earlier positions, where the weights come from comparing queries with keys and the mask hides the future.
3. Residual connections, layer norm, and dropout are what make the network deep enough to train, not part of the core idea.
4. Scale is the main difference between this toy and ChatGPT. Fine-tuning is the main difference between a text completer and an assistant.

Checklist to rebuild it yourself

- [ ] Download Tiny Shakespeare, build `stoi` / `itos`, encode the text into a tensor.
- [ ] Split 90/10 and write `get_batch()` returning inputs and shifted targets.
- [ ] Train the bigram model and confirm loss near 2.5.
- [ ] Reproduce the averaging trick three ways (loop, matrix multiply, softmax with mask) and check they match.
- [ ] Write one `Head` with query, key, value, scaling, and mask.
- [ ] Combine heads into `MultiHeadAttention`, add position embeddings.
- [ ] Add `FeedForward`, residual connections, and layer norm to make a `Block`.
- [ ] Stack 6 blocks, add dropout, scale the settings, and train.
- [ ] Generate text and compare validation loss to about 1.5.

- [ ] Change one setting at a time (heads, layers, context) and note the effect on loss.

Where to go next: *Let's build the GPT Tokenizer* (subword tokens), then *Let's reproduce GPT-2 (124M)*, then nanoGPT and nanochat.